МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра вычислительной техники

ОТЧЕТ

по лабораторной работе №7

по дисциплине «Организация процессов и программирование в среде Linux»

Тема: Обмен данными через канал

Студент гр. 3311 _____ Баймухамедов Р. Р.

Преподаватель _____ Лазарев Э. А.

Санкт-Петербург
2026

ЦЕЛЬ РАБОТЫ

Познакомиться с механизмом обмена данными через программный канал и системными вызовами, обеспечивающими такой обмен.

ЗАДАНИЕ

1. Разработать программу, которая построчно читает входной файл и передаёт его содержимое через канал двум потомкам. Первый потомок записывает нечётные символы в свой файл, второй — чётные. Чередование чтения организовать прямым обменом сигналами SIGUSR1 и SIGUSR2. После окончания записи родитель отправляет обоим потомкам SIGQUIT и ожидает их завершения.

2. Откомпилировать программы, запустить их и вывести содержимое входного и выходных файлов.

1 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Программный канал pipe представляет собой буфер ядра для обмена данными между процессами. Вызов pipe создаёт два файловых дескриптора: channel[0] используется для чтения, channel[1] — для записи. После fork потомки наследуют открытые дескрипторы. Вызов exec заменяет программу процесса; дескрипторы без флага FD_CLOEXEC остаются открытыми.

Канал передаёт поток байтов и не сохраняет границы строк. Функция write записывает данные, read возвращает число прочитанных байтов. При пустом канале и наличии открытого конца записи блокирующее чтение ожидает новые данные. Когда все концы записи закрыты и буфер опустел, read возвращает 0 — конец потока. Поэтому каждый процесс должен закрыть неиспользуемые дескрипторы.

При заполнении канала блокирующая запись ожидает освобождения места. Если все концы чтения закрыты, запись вызывает SIGPIPE. В

данной программе родитель записывает данные в канал, а два потомка поочерёдно читают из него по одному байту.

Сигналы SIGUSR1 и SIGUSR2 используются для передачи очереди чтения между потомками. Они блокируются до fork, а затем принимаются с помощью sigwaitinfo. Маска блокировки наследуется при fork и сохраняется при exec, поэтому сигнал, пришедший до начала ожидания, не завершит процесс по умолчанию. SIGQUIT в данной программе является уведомлением о завершении записи.

2 ПРОГРАММА

2.1 Подготовка входных и выходных файлов

Используется входной файл input.txt; нечётные и чётные байты записываются в odd.txt и even.txt. Исходные файлы программы и исполняемые файлы должны находиться в одном каталоге.

[Создай каталог командой mkdir -p ~/lab7 и перейди в него: cd ~/lab7. Создай файлы common.h, split.c и reader.c, например командой nano common.h. Скопируй в них соответствующие листинги из разделов 2.2–2.4. Для каждого файла используй обычный текстовый редактор.]

[Создай тестовый файл следующей командой:]

```
[printf 'ABCDE\n12345\n' > input.txt]
```

[Выполни cat -n input.txt. Сделай скриншот терминала и вставь его здесь вместо этой подсказки. Остальные подсказки в квадратных скобках перед сдачей удали; квадратные скобки внутри кода оставь.]

Рисунок 1 - Содержимое входного файла

2.2 Общие функции common.h

Заголовочный файл содержит подключение библиотек, обработку ошибок, запись всех переданных байтов и ожидание завершения потомка. Он используется программой родителя и программой потомков.

```
#ifndef COMMON_H
#define COMMON_H
```

```
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

static inline void fail(const char *message)
{
    perror(message);
    exit(EXIT_FAILURE);
}

static inline void write_all(int fd, const void *data, size_t size)
{
    const char *p = data;
    while (size > 0) {
        ssize_t n = write(fd, p, size);
        if (n == -1 && errno == EINTR) continue;
        if (n <= 0) fail("write");
        p += n;
        size -= (size_t)n;
    }
}

static inline int wait_child(pid_t pid)
{
    int status;
    while (waitpid(pid, &status, 0) == -1) {
        if (errno != EINTR) fail("waitpid");
    }
    return WIFEXITED(status) && WEXITSTATUS(status) == 0;
}
#endif
```

2.3 Программа родителя split.c

Родитель блокирует сигналы синхронизации, создаёт канал и двух потомков, затем построчно записывает входной файл в канал. Потомки запускают reader с разными номерами и выходными файлами. После передачи всех строк родитель отправляет SIGQUIT, закрывает свой конец записи и ожидает завершения обоих потомков.

```
#include "common.h"

int main(int argc, char *argv[])
{
    if (argc != 4) {
        fprintf(stderr, "Usage: %s input odd_output even_output\n", argv[0]);
        return EXIT_FAILURE;
    }
    FILE *input = fopen(argv[1], "r");
    if (input == NULL) fail("fopen input");
    sigset_t signals;
    sigemptyset(&signals);
```

```
    sigaddset(&signals, SIGUSR1);
    sigaddset(&signals, SIGUSR2);
    sigaddset(&signals, SIGQUIT);
    if (sigprocmask(SIG_BLOCK, &signals, NULL) == -1) fail("sigprocmask");
    int channel[2];
    if (pipe(channel) == -1) fail("pipe");
    pid_t children[2];
    for (int i = 0; i < 2; ++i) {
        children[i] = fork();
        if (children[i] == -1) fail("fork");
        if (children[i] == 0) {
            close(channel[1]);
            fclose(input);
            char descriptor[32], number[8], peer[32];
            snprintf(descriptor, sizeof descriptor, "%d", channel[0]);
            snprintf(number, sizeof number, "%d", i + 1);
            snprintf(peer, sizeof peer, "%ld",
                     i == 0 ? 0L : (long)children[0]);
            execl("./reader", "reader", descriptor, number, peer,
                  argv[i + 2], (char *)NULL);
            perror("execl reader");
            _exit(EXIT_FAILURE);
        }
    }
    close(channel[0]);
    char *line = NULL;
    size_t capacity = 0;
    ssize_t length;
    while ((length = getline(&line, &capacity, input)) != -1)
        write_all(channel[1], line, (size_t)length);
    if (ferror(input)) fail("getline");
    free(line);
    fclose(input);
    if (kill(children[0], SIGQUIT) == -1) fail("kill SIGQUIT");
    if (kill(children[1], SIGQUIT) == -1) fail("kill SIGQUIT");
    close(channel[1]);
    int first_ok = wait_child(children[0]);
    int second_ok = wait_child(children[1]);
    return first_ok && second_ok ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

2.4 Программа потомков reader.c

Первый потомок ожидает SIGUSR1, читает один байт и передаёт очередь второму сигналом SIGUSR2. Второй ожидает SIGUSR2 и после чтения возвращает очередь сигналом SIGUSR1. Второй потомок знает PID первого и начинает обмен; первый узнаёт PID второго из поля si_pid первого сигнала.

Получение SIGQUIT отмечает окончание записи, но не прерывает чтение: в канале ещё могут оставаться данные. Потомки продолжают передавать очередь до получения EOF. Потомок, обнаруживший EOF,

передаёт последнюю очередь соседу, чтобы тот также завершил чтение. Если SIGQUIT ещё ожидает доставки, он принимается перед завершением.

```
#include "common.h"
#include <fcntl.h>

int main(int argc, char *argv[])
{
    if (argc != 5) return EXIT_FAILURE;
    int fd = atoi(argv[1]);
    int number = atoi(argv[2]);
    pid_t peer = (pid_t)atol(argv[3]);
    int turn = number == 1 ? SIGUSR1 : SIGUSR2;
    int next_turn = number == 1 ? SIGUSR2 : SIGUSR1;
    int output = open(argv[4], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (output == -1) fail("open output");
    sigset_t signals;
    sigemptyset(&signals);
    sigaddset(&signals, turn);
    sigaddset(&signals, SIGQUIT);
    if (number == 2 && kill(peer, SIGUSR1) == -1)
        fail("kill first turn");
    int finished = 0;
    for (;;) {
        siginfo_t info;
        int signal_number;
        do {
            signal_number = sigwaitinfo(&signals, &info);
        } while (signal_number == -1 && errno == EINTR);
        if (signal_number == -1) fail("sigwaitinfo");
        if (signal_number == SIGQUIT) {
            finished = 1;
            continue;
        }
        if (peer == 0) peer = info.si_pid;
        char symbol;
        ssize_t n;
        do {
            n = read(fd, &symbol, 1);
        } while (n == -1 && errno == EINTR);
        if (n == -1) fail("read");
        if (n == 0) {
            if (!finished) {
                sigset_t quit;
                sigemptyset(&quit);
                sigaddset(&quit, SIGQUIT);
                int received;
                int error = sigwait(&quit, &received);
                if (error != 0) { errno = error; fail("sigwait"); }
            }
            if (kill(peer, next_turn) == -1 && errno != ESRCH)
                fail("kill final turn");
            break;
        }
        write_all(output, &symbol, 1);
        if (kill(peer, next_turn) == -1) fail("kill next turn");
    }
    close(fd);
    if (close(output) == -1) fail("close output");
```

```
    return EXIT_SUCCESS;
}
```

3 КОМПИЛЯЦИЯ И ЗАПУСК ПРОГРАММЫ

3.1 Компиляция

Программы компилируются отдельно компилятором GCC. Файл common.h подключается директивой include и отдельно не компилируется.



Рисунок 2 - Компиляция программ

3.2 Запуск и разделение символов

Для проверки разделения символов передаются имя входного файла и имена двух выходных. Нумерация начинается с единицы и продолжается через весь файл. Перевод строки также считается символом.

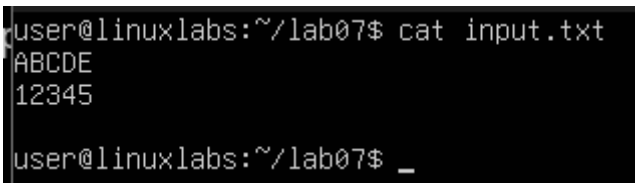


Рисунок 3 - Содержимое input.txt

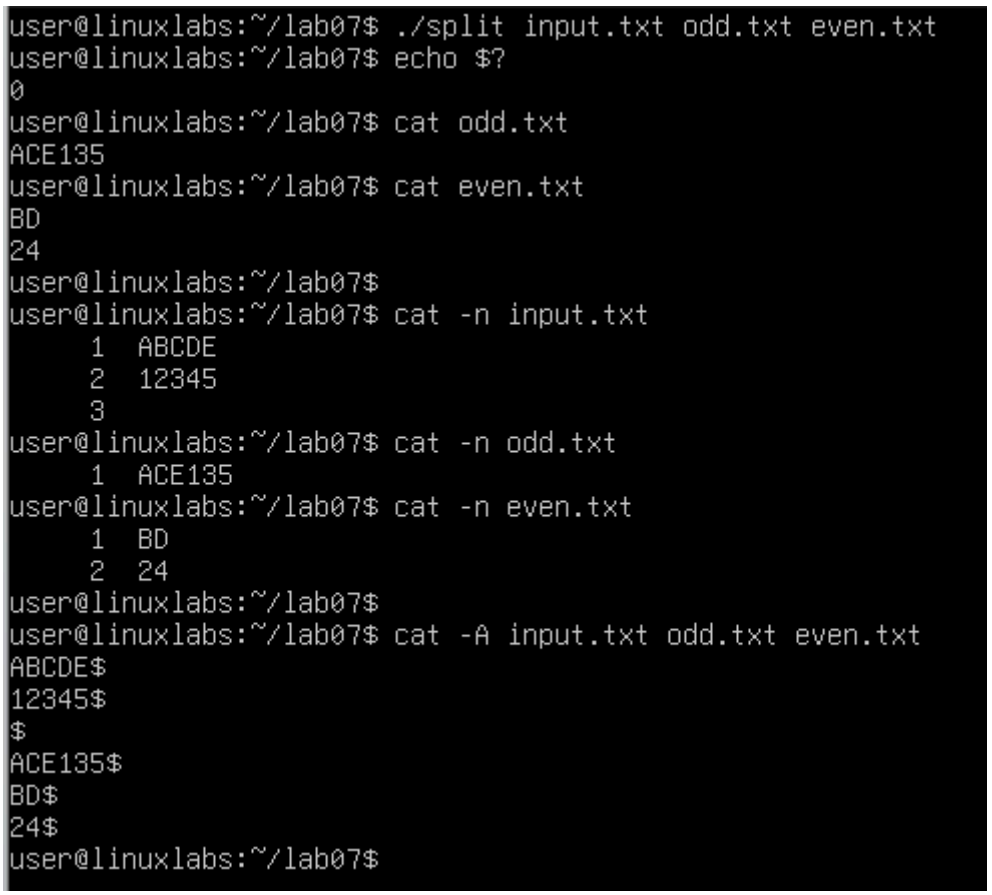


Рисунок 4 - Разделение нечётных и чётных символов

4 АНАЛИЗ РЕЗУЛЬТАТОВ

Очередь чтения передаётся непосредственно между потомками. Поэтому первый читатель получает байты с позициями 1, 3, 5 и так далее, второй — 2, 4, 6 и так далее. Для входных данных ABCDE\n12345\n результат первого читателя составляет ACE135, второго — BD\n24\n, где \n обозначает байт перевода строки.

EOF возникает только после закрытия всех концов записи и извлечения оставшихся данных из канала. Родитель закрывает конец записи после SIGQUIT, а потомки закрывают свои концы чтения после EOF. Такое закрытие необходимо для корректного завершения и предотвращения бесконечного ожидания.

Программы обрабатывают байты. Для однобайтового текста это соответствует разделению символов, включая пробелы и переводы строк.

В UTF-8 русские буквы занимают несколько байтов, поэтому разделение по чётности байтов может сделать выходные файлы некорректными текстами. Для проверки выбран текст из символов ASCII.

[После выполнения проверь соответствие анализа своим скриншотам. Если использовал другие входные данные, замени пример результатов. Скриншоты должны быть получены на твоём компьютере.]

ВЫВОД

Разработана программа разделения нечётных и чётных байтов через канал. Изучены pipe, read, write и close. Чередование чтения обеспечивают SIGUSR1 и SIGUSR2; об окончании записи сообщает SIGQUIT.